03/03/2026

Che cos'e un algoritmo?

1. Algoritmi: li usi gia (senza saperlo)

Ogni mattina fai piu o meno le stesse cose: ti svegli, ti alzi, ti vesti, fai colazione, esci di casa. Congratulazioni: hai appena eseguito un **algoritmo**. Non servono computer o formule strane: un algoritmo e semplicemente una **sequenza ordinata di passi** che porta a un risultato.

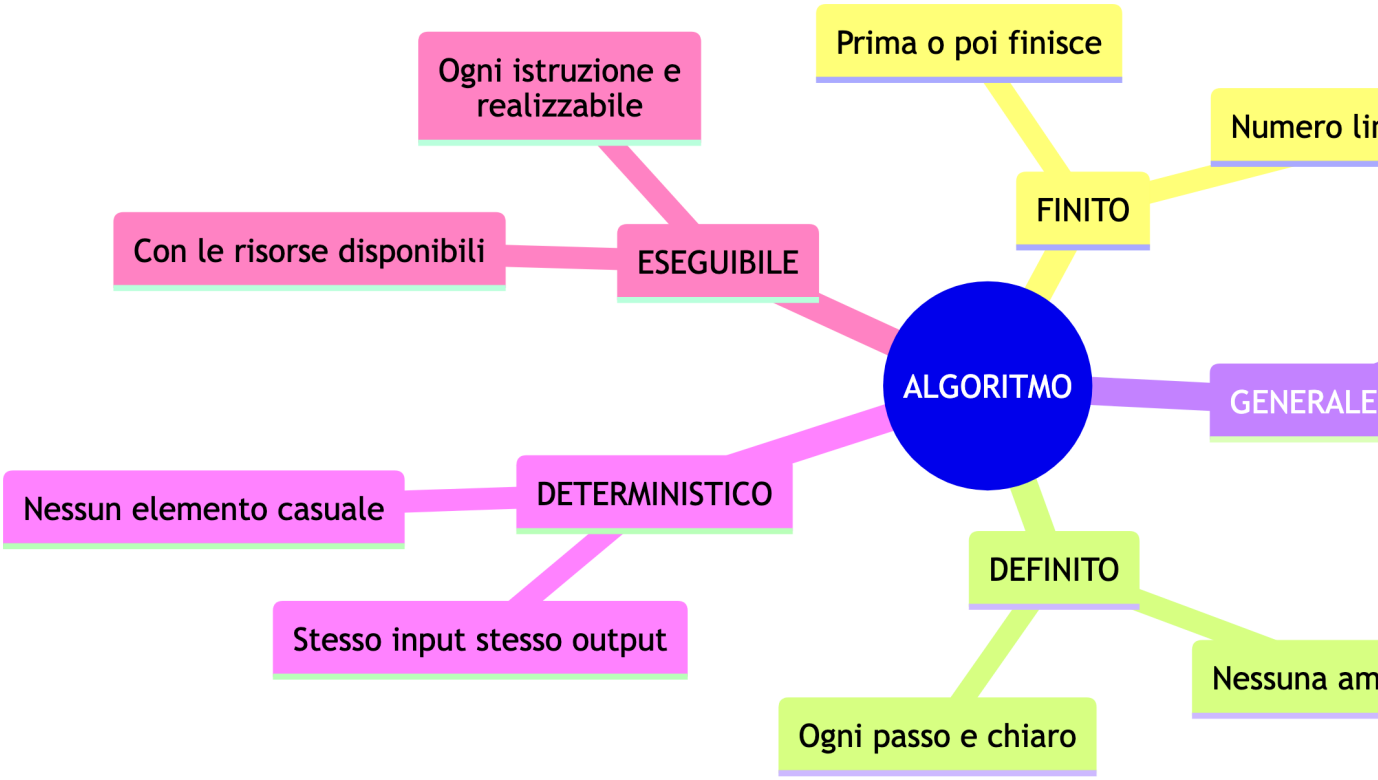
> **Definizione:** Un **algoritmo** e una sequenza finita, ordinata e non ambigua di istruzioni che, partendo da dati iniziali (input), produce un risultato (output) in un tempo finito.

Tradotto in italiano corrente: e una «ricetta» precisa per risolvere un problema. Tipo le istruzioni IKEA, ma scritte bene.

---

2. Le 5 proprieta fondamentali

Perche una sequenza di passi possa chiamarsi algoritmo, deve avere queste proprieta:



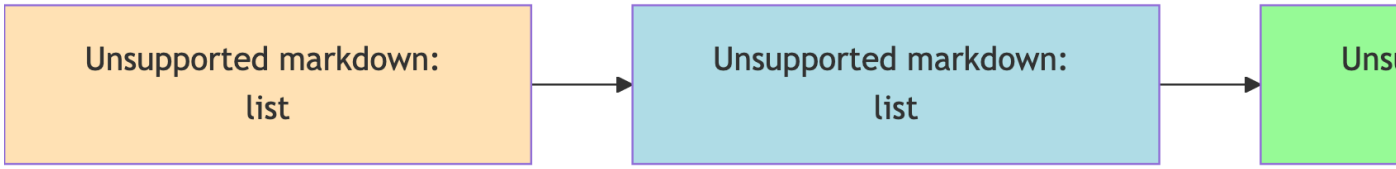
Proprieta	Significato	Esempio
Finito	Ha un numero limitato di passi e prima o poi finisce	Una ricetta ha un numero preciso di passaggi
Definito	Ogni passo e descritto in modo chiaro e non ambiguo	«Aggiungi 200g di farina» non «aggiungi un po' di farina»
Generale	Risolve una **classe** di problemi, non un singolo caso	La formula dell'area del rettangolo funziona per TUTTI i rettangoli
Deterministico	A parita di input produce sempre lo stesso output	2 + 3 fa sempre 5, non a volte 5 e a volte 7
Eseguibile	Ogni istruzione puo essere effettivamente svolta	«Calcola la radice quadrata» si puo fare, «leggi nel pensiero» no

---

3. Dall'idea all'algoritmo: il pensiero computazionale

Risolvere un problema con un algoritmo richiede un modo di pensare particolare, che si chiama **pensiero computazionale**. Non e una roba solo da nerd: e un modo strutturato di affrontare qualsiasi problema.

Il pensiero computazionale si basa su 4 pilastri:



Scomposizione

Prendi un problema grande e spezzalo in problemi piu piccoli e gestibili.

Esempio: «Prepara una carbonara» diventa:

1. Cuoci la pasta
2. Prepara il condimento (uova + pecorino + pepe)
3. Cuoci il guanciale
4. Unisci tutto

Ogni sotto-problema e piu facile da affrontare singolarmente.

Riconoscimento di pattern
Cerca schemi che si ripetono. Se hai gia risolto un problema simile, puoi riutilizzare la soluzione.

Esempio: Calcolare l'area di un rettangolo e calcolare l'area di un parallelogramma usano la stessa formula (base x altezza). Il pattern e lo stesso!

Astrazione
Concentrati su quello che conta ed elimina i dettagli inutili.

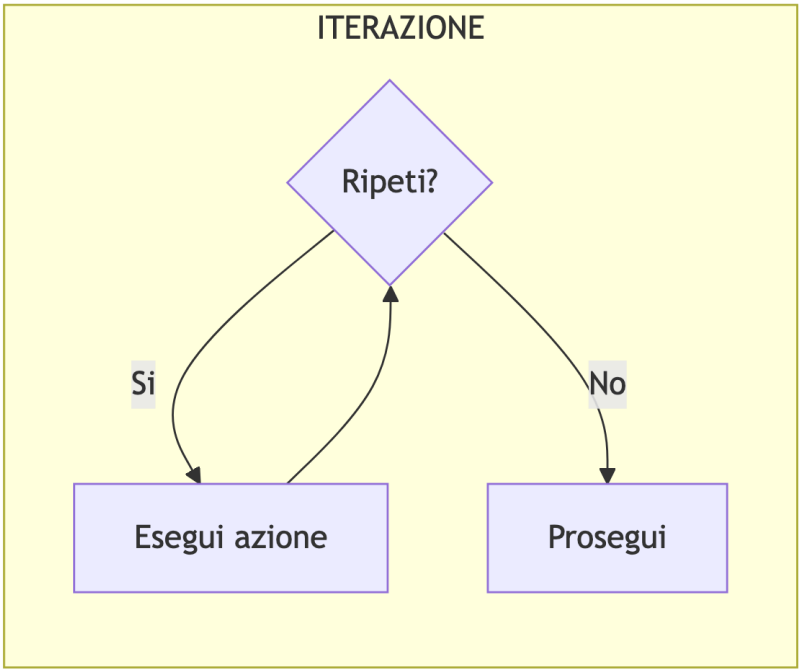
Esempio: Se devi calcolare il percorso piu breve per andare a scuola, non ti importa del colore delle case lungo la strada. Ti servono solo le distanze e le direzioni.

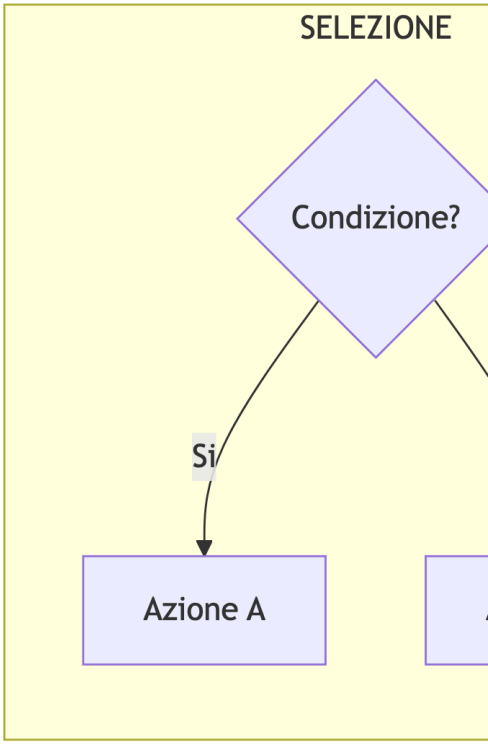
Progettazione algoritmica
Scrivi la sequenza di passi che risolve il problema.

---

4. Le strutture fondamentali

Qualsiasi algoritmo, dal piu banale al piu complesso, si costruisce combinando solo **tre strutture** fondamentali. Tre. Non cento, non mille. Tre. Questo e il bello dell'informatica.





Sequenza
Le istruzioni vengono eseguite **una dopo l'altra**, nell'ordine in cui sono scritte. Come leggere un libro: pagina 1, poi pagina 2, poi pagina 3.

Esempio nella vita reale:

1. Apri il rubinetto
2. Bagna le mani
3. Metti il sapone
4. Strofina
5. Sciacqua
6. Chiudi il rubinetto
7. Asciuga le mani

Selezione (o decisione)
Il flusso prende strade diverse in base a una **condizione**. E il bivio: «Se piove prendo l'ombrello, altrimenti esco cosi».

Esempio nella vita reale:

- **SE** la sveglia suona alle 7:00 **ALLORA** mi alzo
- **ALTRIMENTI** dormo ancora 5 minuti (e poi altri 5, e poi altri 5…)

Iterazione (o ciclo)
Un blocco di istruzioni viene **ripetuto** piu volte, finche una condizione e vera (o diventa falsa). Tipo quando studi per un'interrogazione: rileggi gli appunti finche non li sai.

Esempio nella vita reale:

- **FINCHE** il piatto non e pulito, **CONTINUA** a lavarlo
- **RIPETI** «dai un colpo di spugna» **FINCHE** non e tutto splendente

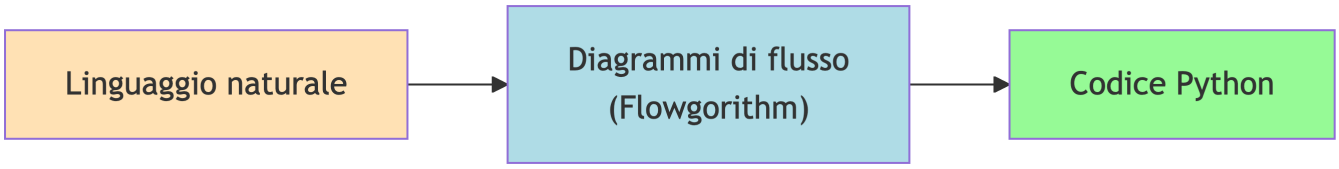
---

5. Rappresentare un algoritmo

Un algoritmo puo essere rappresentato in diversi modi. Ognuno ha i suoi vantaggi:

Rappresentazione	Cos'e	Pro	Contro
Linguaggio naturale	Descrizione a parole	Facile da capire	Puo essere ambiguo
Pseudocodice	Via di mezzo tra parole e codice	Preciso ma leggibile	Non eseguibile
Diagramma di flusso	Schema grafico con forme	Visuale e intuitivo	Ingombrante per algoritmi complessi
Codice	Programma vero e proprio	Eseguibile dal computer	Richiede conoscenza del linguaggio

In questo corso seguirai un percorso graduale:



Prima impari a **pensare** l'algoritmo (in italiano), poi a **disegnarlo** (con Flowgorithm), infine a **scriverlo** (in Python).

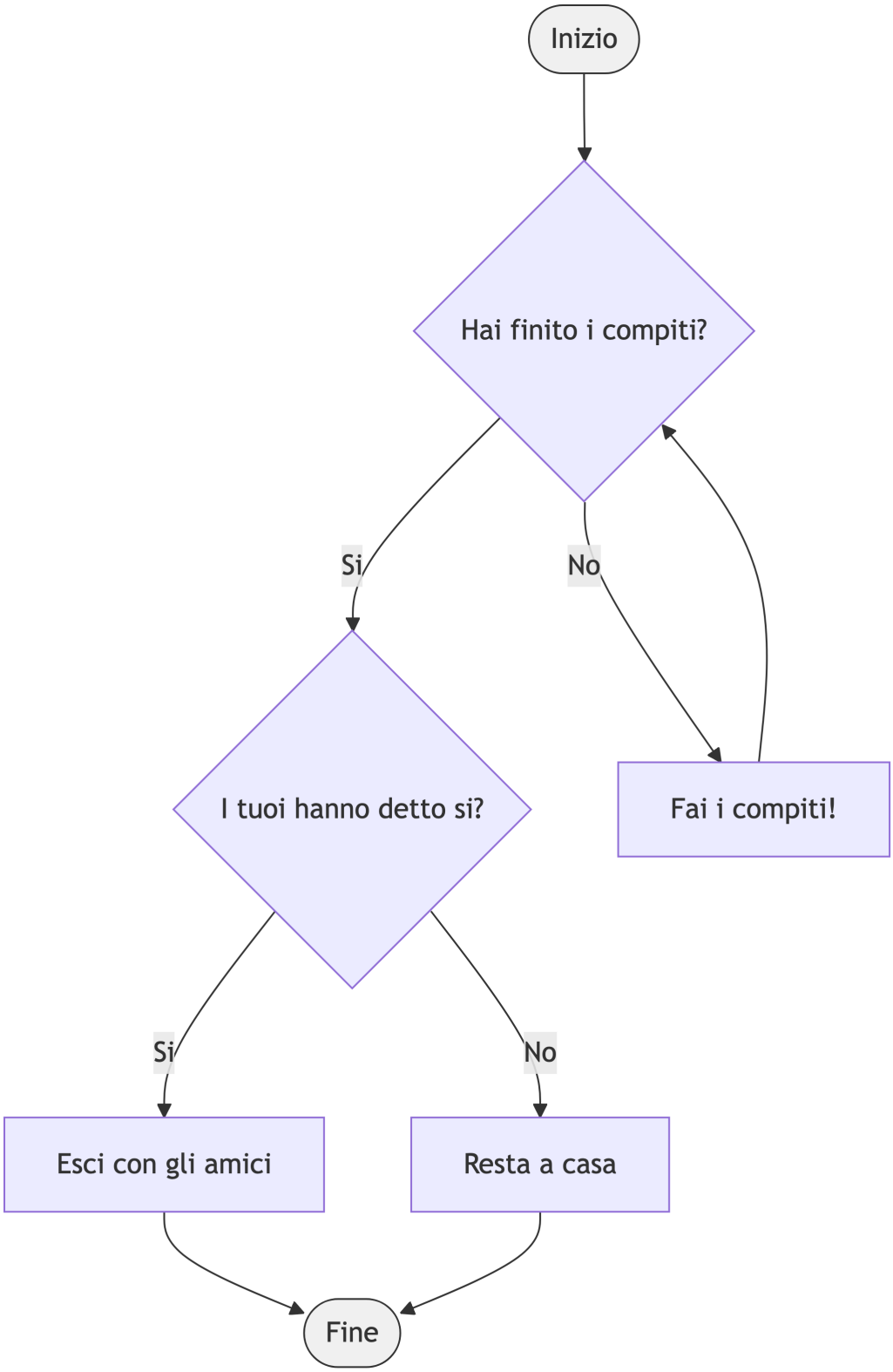
---

6. I diagrammi di flusso

I diagrammi di flusso (flowchart) usano **forme geometriche standard** per rappresentare i diversi tipi di istruzione:

Forma	Nome	Significato
Ovale	Terminatore	Inizio / Fine del programma
Parallelogramma	Input/Output	Lettura dati o stampa risultati
Rettangolo	Elaborazione	Calcoli e assegnazioni
Rombo	Decisione	Condizione vera/falsa
Freccia	Flusso	Direzione dell'esecuzione

Ecco un esempio completo: un algoritmo che decide se puoi uscire la sera.


Inizio
Hai finito i compiti?
Si
No
I tuoi hanno detto si?
Fai i compiti!
Si
No
Esci con gli amici
Resta a casa
Fine

Noti le tre strutture fondamentali in azione?

- **Sequenza**: il flusso va dall'alto verso il basso
- **Selezione**: i rombi con le domande (bivii)
- **Iterazione**: il ciclo «Fai i compiti!» che torna alla domanda

---

7. Pseudocodice: la via di mezzo

Lo pseudocodice e un modo di scrivere un algoritmo che sembra un linguaggio di programmazione, ma e scritto in modo che chiunque possa capirlo. Non ha regole rigide, l'importante e che sia **chiaro e non ambiguo**.

Esempio: L'algoritmo «posso uscire?» in pseudocodice:

```
INIZIO
    RIPETI
        SE compiti_finiti = NO ALLORA
            Fai i compiti
        FINE SE
    FINCHE compiti_finiti = SI

    CHIEDI permesso ai genitori

    SE permesso = SI ALLORA
        STAMPA "Esci con gli amici!"
    ALTRIMENTI
        STAMPA "Resta a casa"
    FINE SE
FINE
```

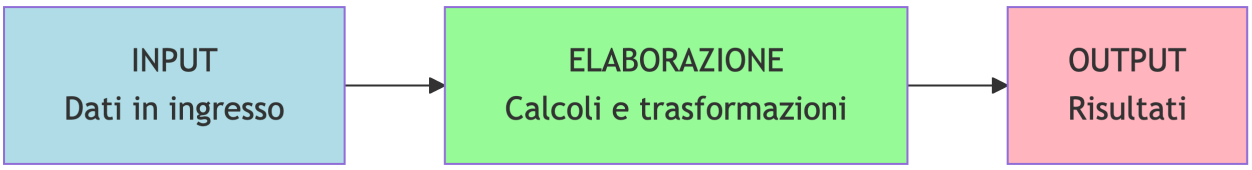
Le parole chiave usate nello pseudocodice sono:

- **INIZIO / FINE** per delimitare il programma
- **SE / ALLORA / ALTRIMENTI / FINE SE** per le decisioni
- **RIPETI / FINCHE** per i cicli
- **LEGGI** per l'input
- **STAMPA** per l'output

---

8. Input, Elaborazione, Output: il modello IPO

Ogni algoritmo segue lo stesso schema fondamentale, chiamato modello **IPO**:



- **Input**: i dati che servono per partire (li chiedi all'utente o li leggi da qualche parte)
- **Elaborazione**: cosa fai con quei dati (calcoli, confronti, trasformazioni)
- **Output**: il risultato finale che mostri all'utente

Esempio concreto:

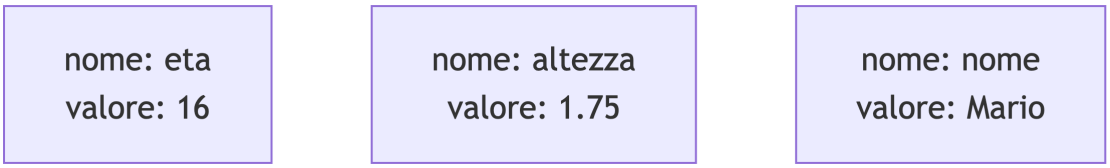
	Ricetta	Algoritmo «area rettangolo»
Input	Ingredienti	Base e altezza
Elaborazione	Cucinare	area = base x altezza
Output	Piatto pronto	Il valore dell'area

Questo modello vale **sempre**: che tu stia facendo una somma o progettando un videogioco, il pattern e lo stesso.

---

9. Le variabili: le scatole della memoria

Quando un algoritmo lavora con dei dati, deve **salvarli da qualche parte**. Le variabili sono come scatole etichettate: ognuna ha un **nome** e contiene un **valore**.



Le variabili hanno un **tipo** che determina cosa possono contenere:

Tipo	Cosa contiene	Esempio
Intero	Numeri senza virgola	42, −7, 0
Reale	Numeri con virgola	3.14, −2.5, 0.001
Stringa	Testo	«Ciao», «Mario Rossi»
Booleano	Vero o Falso	true, false

Regole d'oro per le variabili:

- Dai nomi **chiari e descrittivi** (`eta` e meglio di `x`)
- Dichiara il tipo **prima** di usarle
- Una variabile puo **cambiare valore** durante l'esecuzione (per questo si chiama «variabile»!)

---

10. Esempio completo: dalla vita reale all'algoritmo

Problema: Calcola la media dei voti di un compito in classe.

In linguaggio naturale

1. Chiedi quanti studenti ci sono
2. Per ogni studente, chiedi il voto
3. Somma tutti i voti
4. Dividi la somma per il numero di studenti
5. Mostra la media

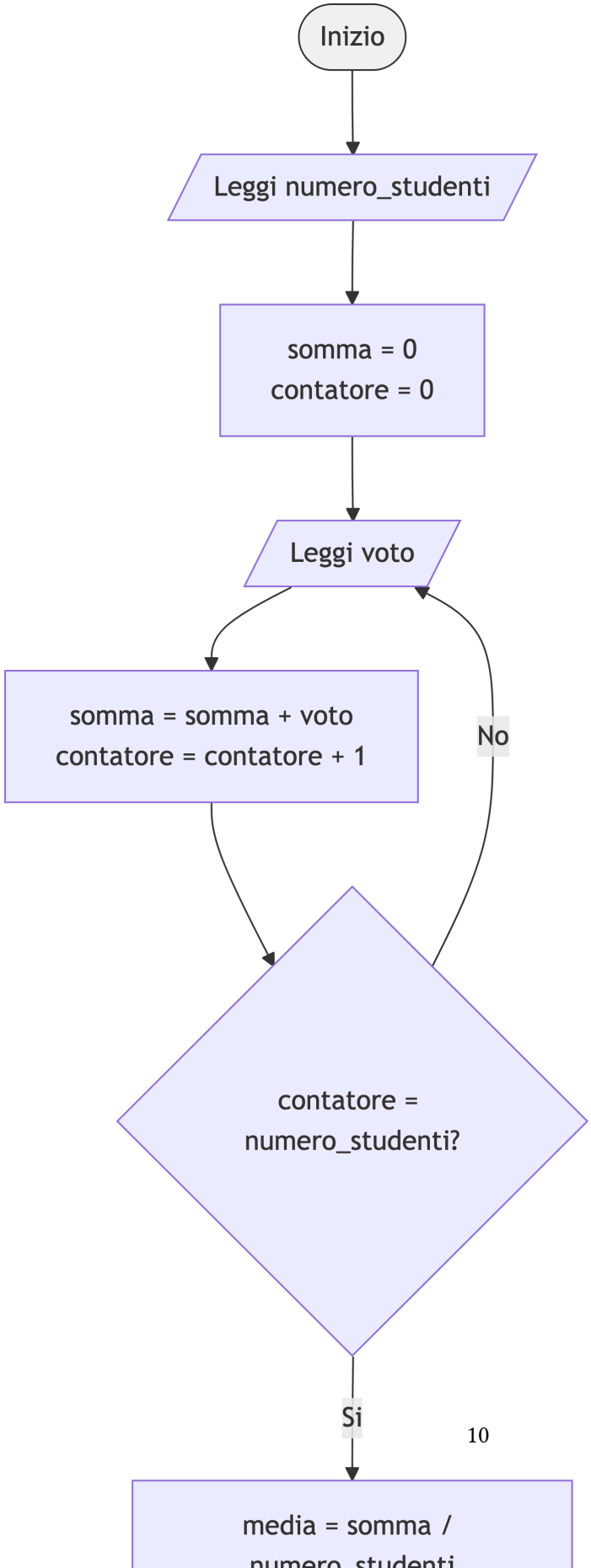
In pseudocodice

```
INIZIO
    LEGGI numero_studenti
    somma = 0
    contatore = 0

    RIPETI
        LEGGI voto
        somma = somma + voto
        contatore = contatore + 1
    FINCHE contatore = numero_studenti

    media = somma / numero_studenti
    STAMPA "La media e: " + media
FINE
```

In diagramma di flusso


Inizio
Leggi numero_studenti
somma = 0
contatore = 0
Leggi voto
somma = somma + voto
contatore = contatore + 1
No
contatore =
numero_studenti?
Si
media = somma /
numero_studenti

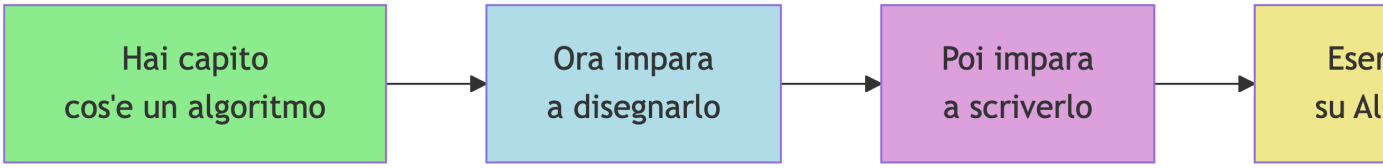
Nota come ritroviamo tutte le strutture: **sequenza** (il flusso dall'alto in basso), **iterazione** (il ciclo che legge i voti), e nel prossimo capitolo aggiungeremo le **selezioni** con Flowgorithm!

---

11. Cosa viene dopo?

Ora che conosci le basi teoriche degli algoritmi, sei pronto per metterle in pratica:

1. **Flowgorithm** - Imparerai a disegnare algoritmi con un ambiente visuale, senza scrivere codice
2. **Python** - Tradurrai quei diagrammi in un vero linguaggio di programmazione
3. **AlgoWeb** - Potrai esercitarti con problemi algoritmici direttamente nel browser



Il bello e che la logica e **sempre la stessa**: cambia solo lo strumento. Se capisci il concetto ora, Flowgorithm e Python saranno solo questione di sintassi.

> Metti in pratica quello che impari
>
> Su AlgoWeb trovi esercizi di coding Python pensati per accompagnarti passo dopo passo: dai problemi piu semplici fino alle sfide avanzate. Puoi scrivere ed eseguire codice direttamente nel browser, senza installare nulla!